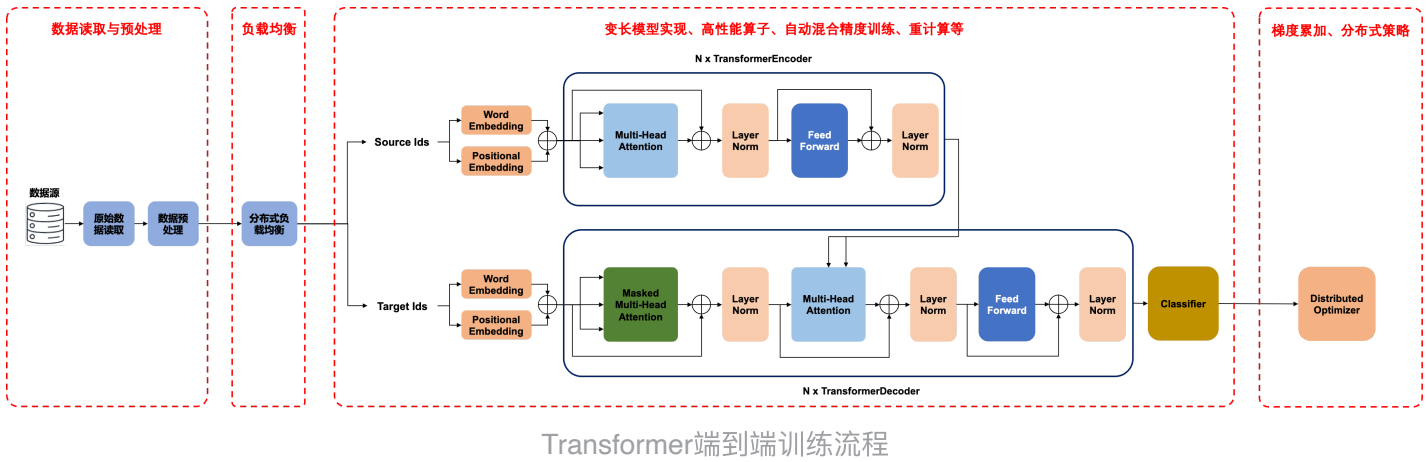


Transformer类模型训练加速方法

下图所示的是Transformer模型端到端训练流程。在训练开始前，首先需要从原始数据源读入训练数据，并进行分词等预处理操作，数据读取与与处理的速度将直接影响模型的训练效率；然后，对分布式训练场景下，需要对输入数据进行动态负载均衡处理，提高分布式训练加速比；在模型前反向运行阶段，飞桨支持高效的变长模型实现，通过高性能算子、自动混合精度训练等优化方式提高基础训练性能，并使用重计算等技术降低模型的显存占用；在分布式训练通信和参数更新阶段，飞桨支持梯度累加等策略提高单卡batch size的同时降低通信量，并提供了高度优化的分组参数切片并行等分布式训练策略，在某些场景下可实现超线性的训练加速效果。

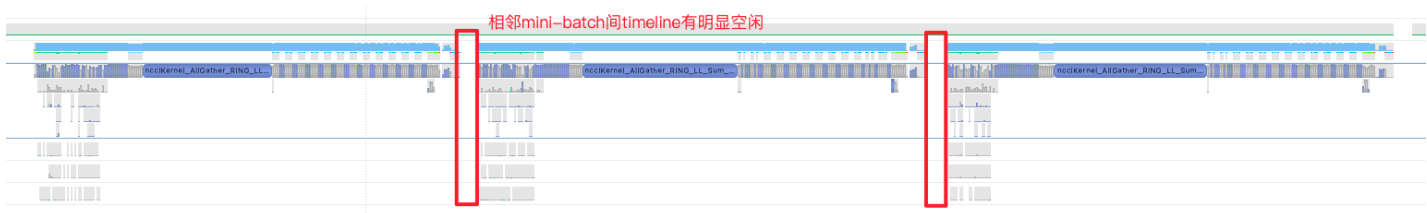


以下具体介绍飞桨的Transformer类模型端到端训练加速方法。

一、数据读取与预处理加速方法

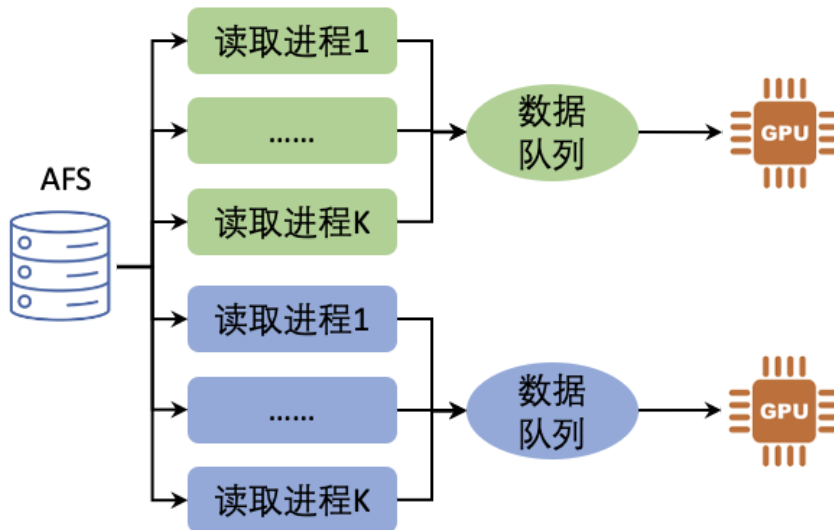
1. 数据读取加速

在实际训练过程中，数据集往往会非常大，甚至有的时候会因为数据量大必须保存在AFS等地方而非本地，数据读取的速度可能会跟不上模型训练速度。在这种情况下，Profile结果的timeline上会明显看到相邻mini-batch之间有明显的空闲时间。



以CLIP模型为例说明由于数据读取速度慢导致Timeline空闲

- Python多进程数据读取加速：这种情况往往需要使用多进程技术进行加速，一种常见的多进程加速技术如下图所示。假设有N个卡参与训练，则每个卡各分配K个数据读取的进程和一个数据队列，每个进程均连接AFS并发读取数据，并将读取后的数据push至每个卡独占的数据队列中。



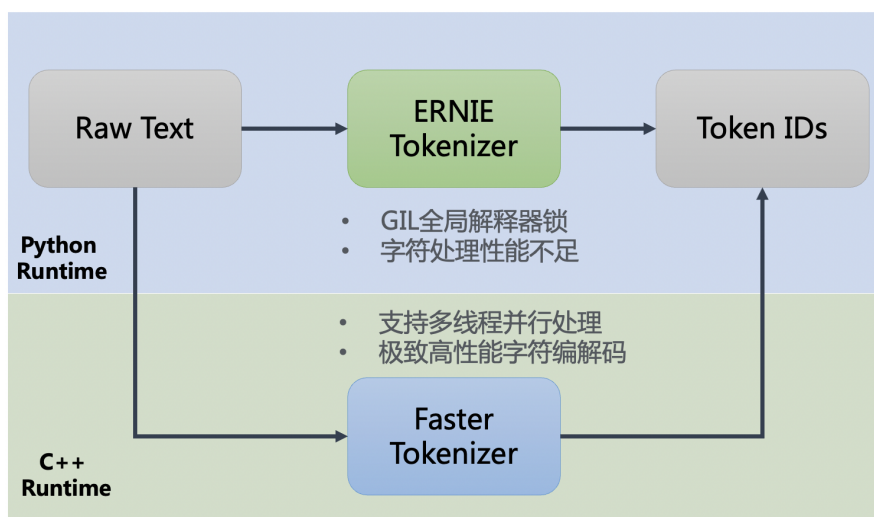
多进程 + 独占队列的方式实现数据读取加速

- C++数据读取加速：若Python数据读取速度仍跟不上模型训练速度，在极致情况下可将Python数据读读取代码改写为C++代码进行加速。但这种方式往往需要较高的编程门槛，一般适用于对性能要求非常高的场景，比如MLPerf等。

2. 数据预处理加速

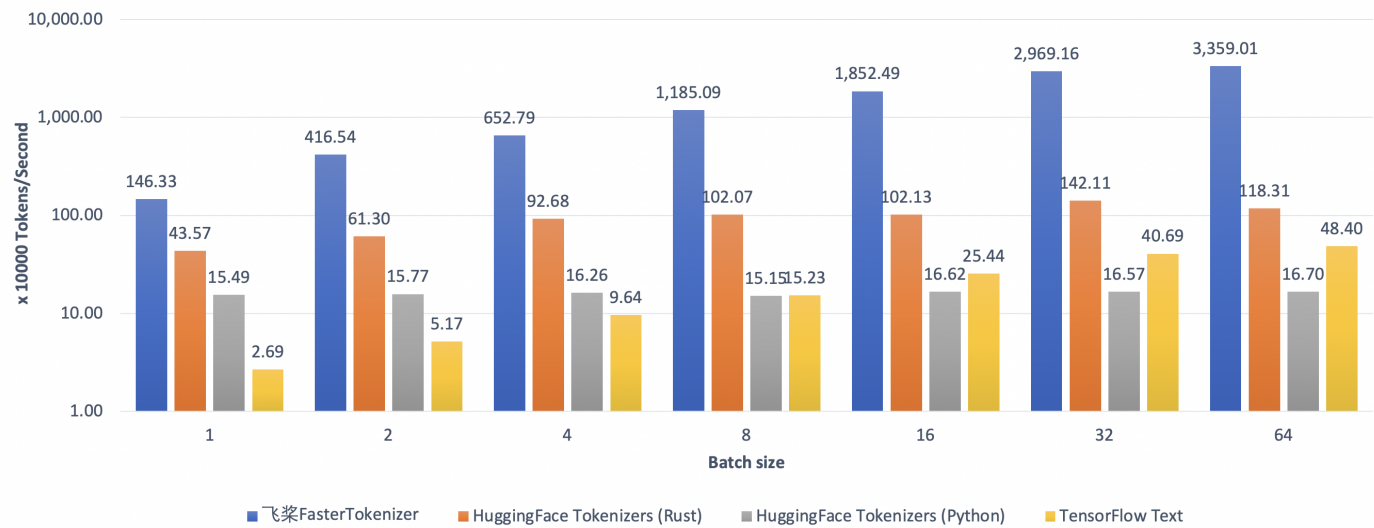
在深度学习训练过程中，往往需要对原始数据数据进行预处理操作后才进入到模型训练。

对于自然语言语料，往往需要经过分词操作转换为Token ID才能输入到网络进行训练。飞桨提供了FasterTokenizer C++ OP 实现快速分词，且可与推理实现全流程打通，便于用户快速部署。



飞桨FasterTokenizer流程图

不同框架文本处理性能对比

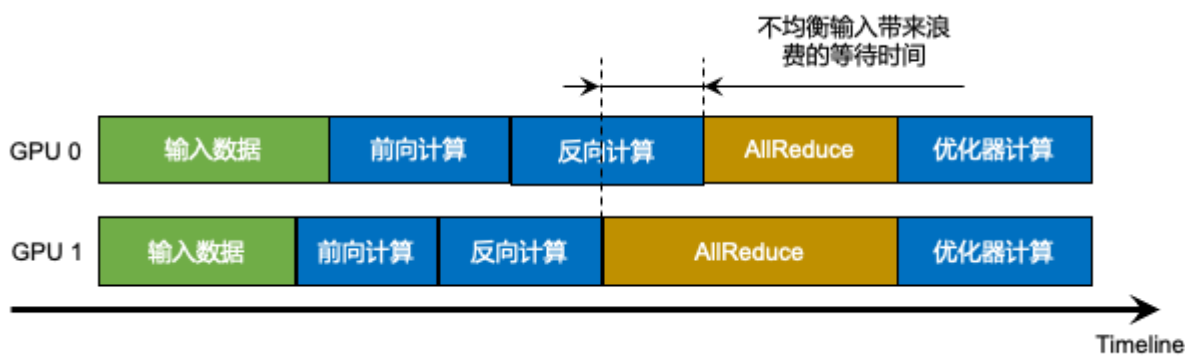


飞桨FasterTokenizer与其他Tokenizer的性能对比

对于多模态模型中的图像输入数据，往往需要经过加噪、随机翻转、随机剪裁、缩放等数据增强操作才能输入到模型进行训练。NVIDIA的DALI框架实现了飞桨框架支持，用户可直接使用DALI提供的图像数据预处理接口实现飞桨模型训练加速。具体可参考[DALI文档](#)。

二、负载均衡方法

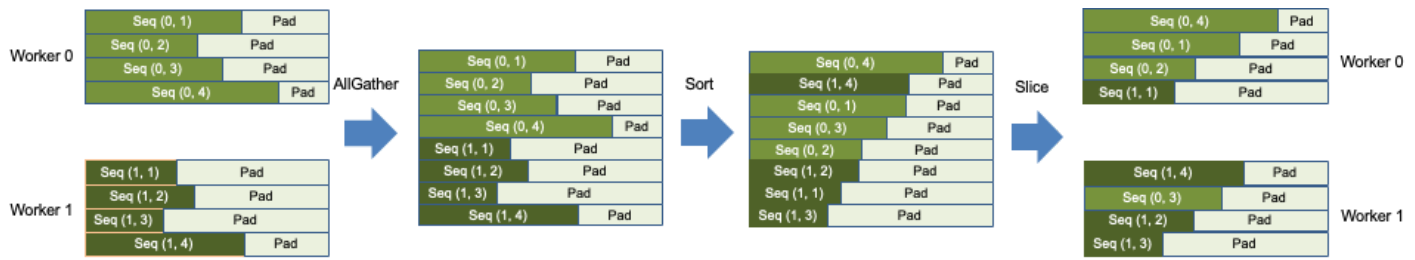
在分布式训练过程中，各卡间的负载均衡严重影响分布式训练效率的提升。下图是数据并行场景下负载不均的示意图，各卡间的负载不均衡导致AllReduce等通信时间变长，严重制约多卡加速比的提升。



数据并行场景下各卡训练步调不协调导致的短板效应

一般而言，分布式训练过程的负载不均问题往往是由于各卡输入数据的不等长，如自然语言语料的序列长度不相等。解决此类问题的思路一般有2类：（1）在训练之前，对原始数据集进行排序处理。这种方法需要在训练前期做大量的数据处理操作。（2）在训练过程中，做动态的数据排序和分发处理。这种方法需要在训练前期做任何数据处理操作，但会在运行期引入额外的计算和通信。

以下以MLPerf BERT模型为例，说明运行期输入数据负载均衡的实现方式。MLPerf BERT模型的原始输入数据是带padding信息的，我们首先将各卡的输入数据进行AllGather操作，并根据实际序列长度（不含padding）对AllGather后的数据进行重排和重新分发，动态实现输入数据的负载均衡。



MLPerf BERT模型运行期输入数据负载均衡方案

为了减少运行期负载均衡引入的额外机器开销，飞桨创新提出了异构设备混合负载均衡方案：

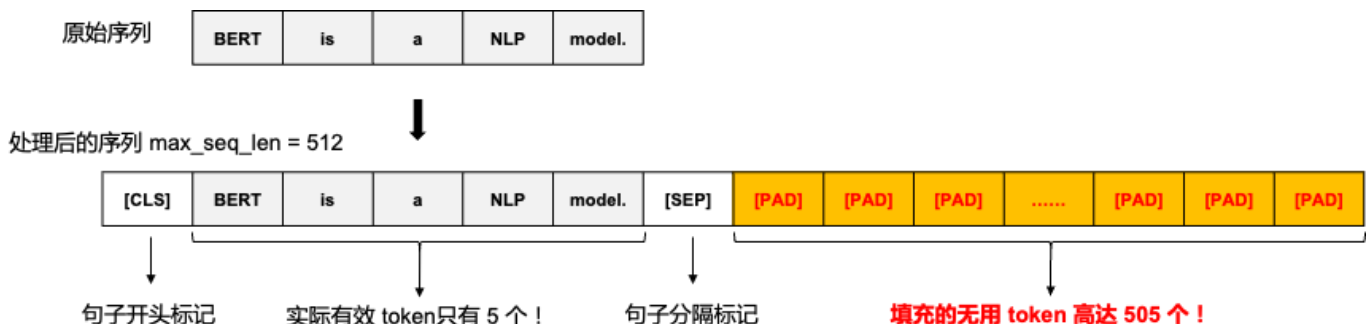
- 在训练初期的 warmup 阶段，数据队列处于饥饿状态，需要快速准备出训练数据启动训练。此时，可使用 GPU NV-LINK + IB 高带宽通信的方式进行负载均衡操作。
- 在训练稳定阶段，模型训练需要占用大量的GPU资源。此时可使用空闲的CPU设备进行MPI通信，解放GPU资源，实现CPU负载均衡操作与GPU模型训练过程的完全重叠。



飞桨的异构设备负载均衡方案

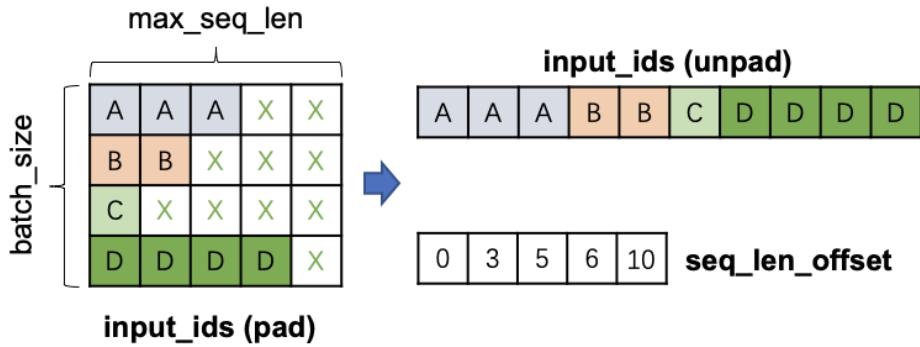
三、变长模型实现加速

自然语言处理的原始输入序列大多是变长的。为了组网和计算方便，往往会通过padding的方式将每个序列填充为等长的长度，但带来了冗余数据（显存资源浪费）和冗余计算（计算资源浪费）。



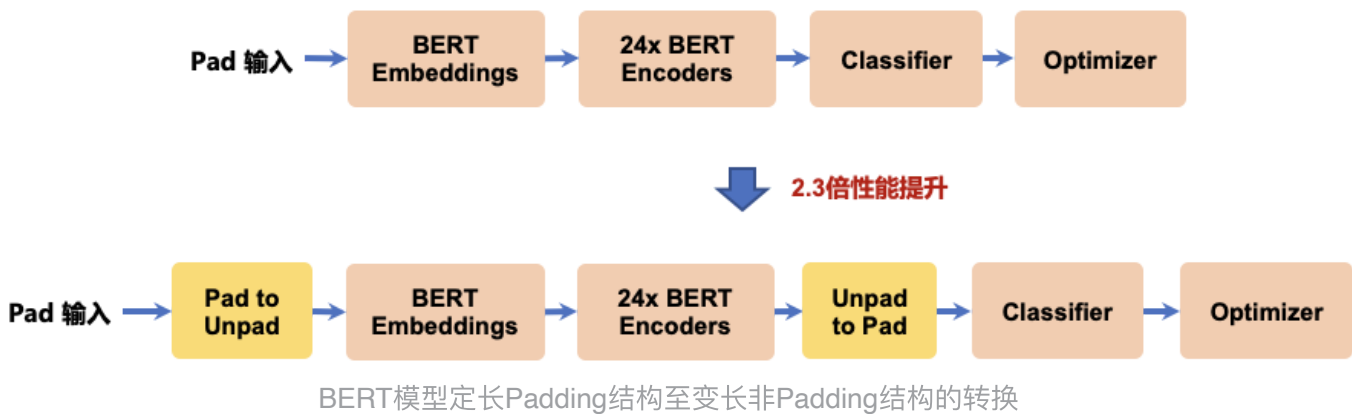
自然语言原始序列padding填充示意

飞桨提供了针对Transformer类模型的高效变长网络支持：在数据输入端，将输入数据的 `batch_size` 维和 `max_seq_len` 维合并为一维，去除填充的无用 token，记录每个序列的有效长度信息；在模型实现侧，飞桨针对Transformer类模型实现了高效的变长算子实现，涵盖Multi-Head Attention、Feed Forward Network等模块。以MLPerf BERT模型为例，变长模型带来了2.3倍的性能提升。



实际序列长度： $\text{seq_len}[i] = \text{seq_len_offset}[i + 1] - \text{seq_len_offset}[i]$

定长Padding数据到变长非Padding数据的转换

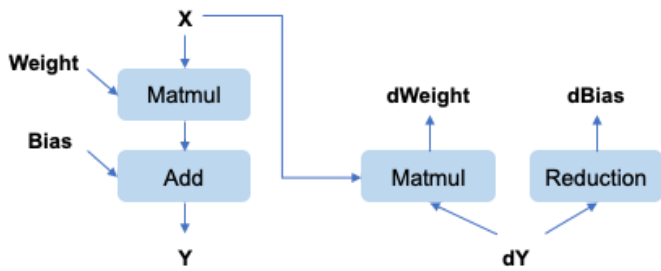


BERT模型定长Padding结构至变长非Padding结构的转换

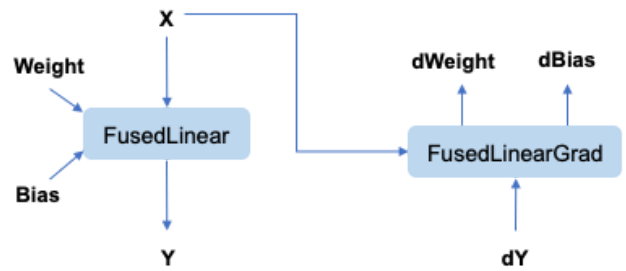
四、高性能算子加速

Transformer类模型最重要的两类结构是Multi-Head Attention和Feed Forward Network模块。飞桨针对此类结构进行了大量深度优化，包括：

- **Linear Fusion：** 将Multi-Head Attention和Feed Forward Network中Linear层的矩阵乘法和偏置加法进行融合，节省访存开销。此外，还支持与ReLU、GeLU等激活层的融合，进一步提升训练性能。

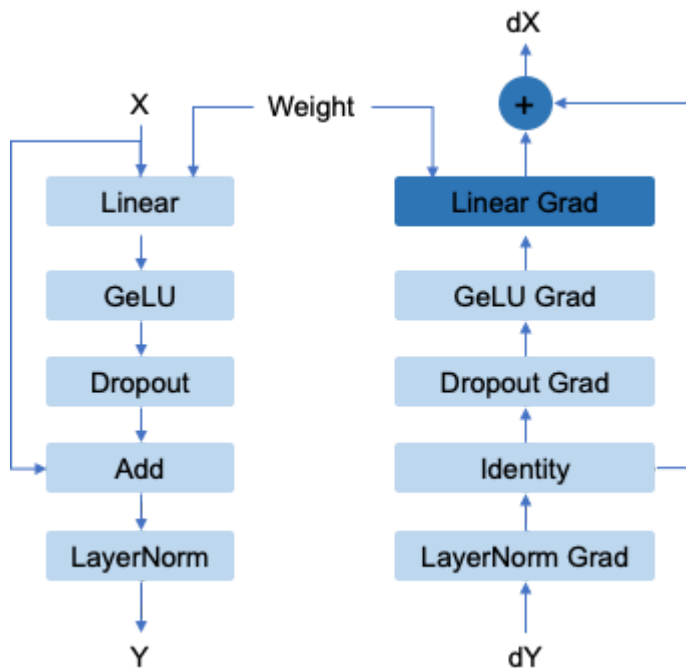


融合前的全连接层前反向

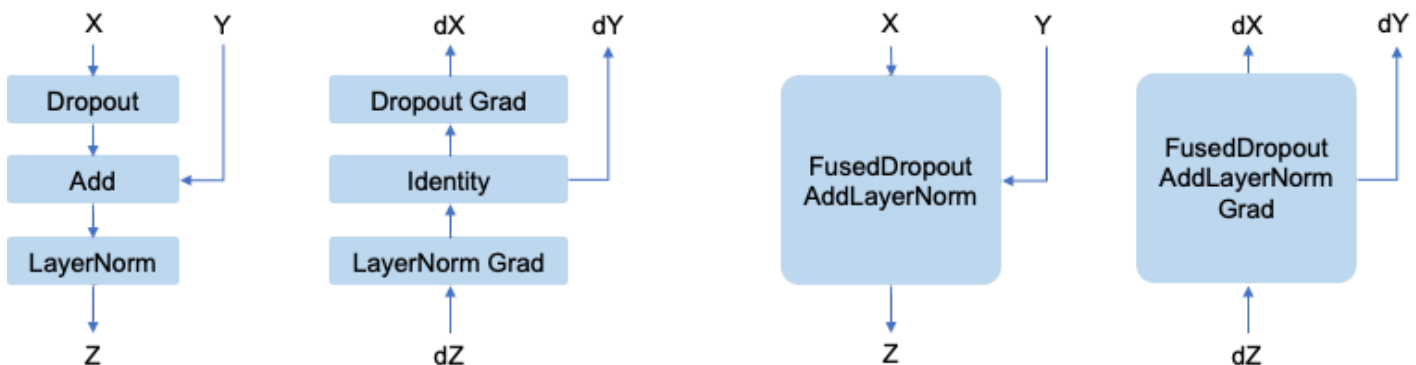


融合后的全连接层前反向

- Inplace Addto Fusion: 在Residual Add等残差网络结构中, 实现Linear反向矩阵乘法与Residual Add梯度累加的融合, 减少OP数量和仿存开销。



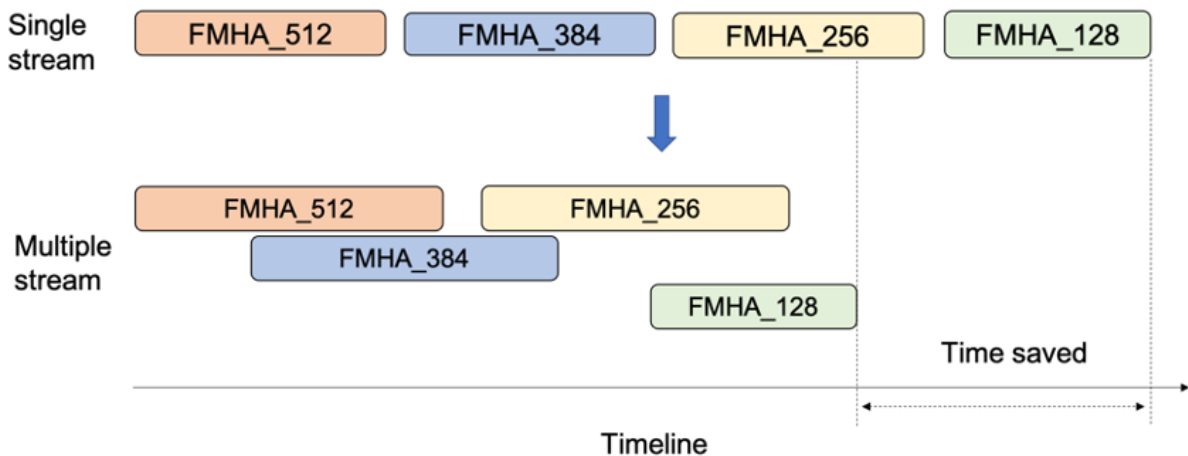
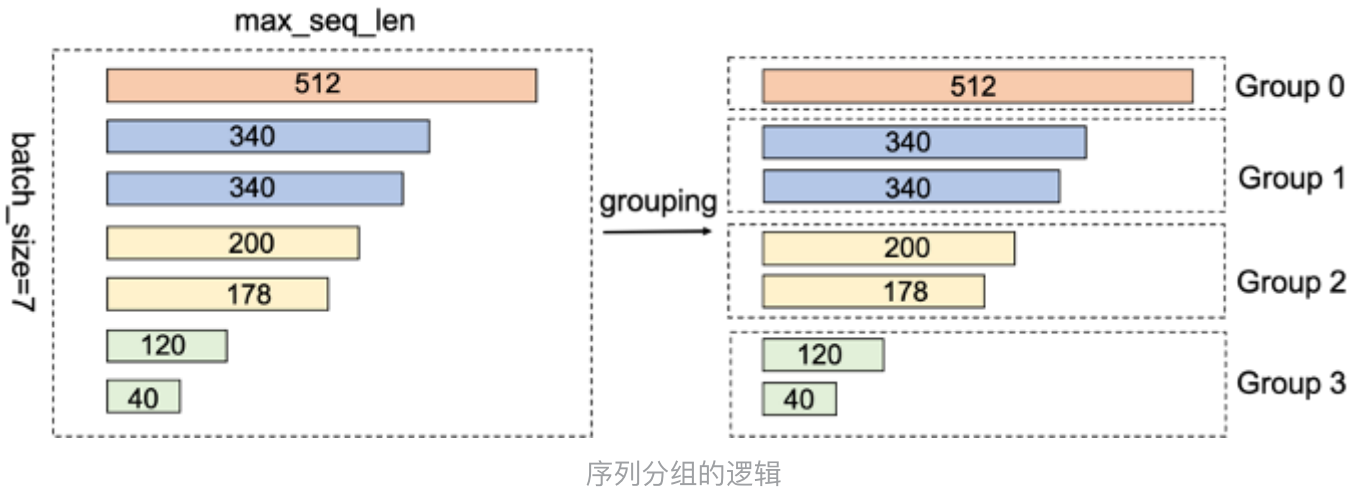
- Dropout + Add + LayerNorm融合: 将Dropout、Add两个elementwise操作的OP与LayerNorm进行融合, 减少OP数量和仿存开销。



上述优化策略已通过飞桨的 `FusedLinear`、`FusedAttention` 和 `FusedFeedForward` API的形式提供给用户, 开箱即用。

此外，针对MLPerf等极致性能场景，飞桨还实现了变长Attention的高效算子支持：首先，按照序列有效长度进行分组；然后每个分组对应一个最优的CUDA Kernel

实现，使用多CUDA Stream并行执行多个组的计算逻辑。



飞桨还计划引入FlashAttention、Memory Efficient Attention等优化策略，给用户提供极致的训练性能体验。

五、自动混合精度训练技术

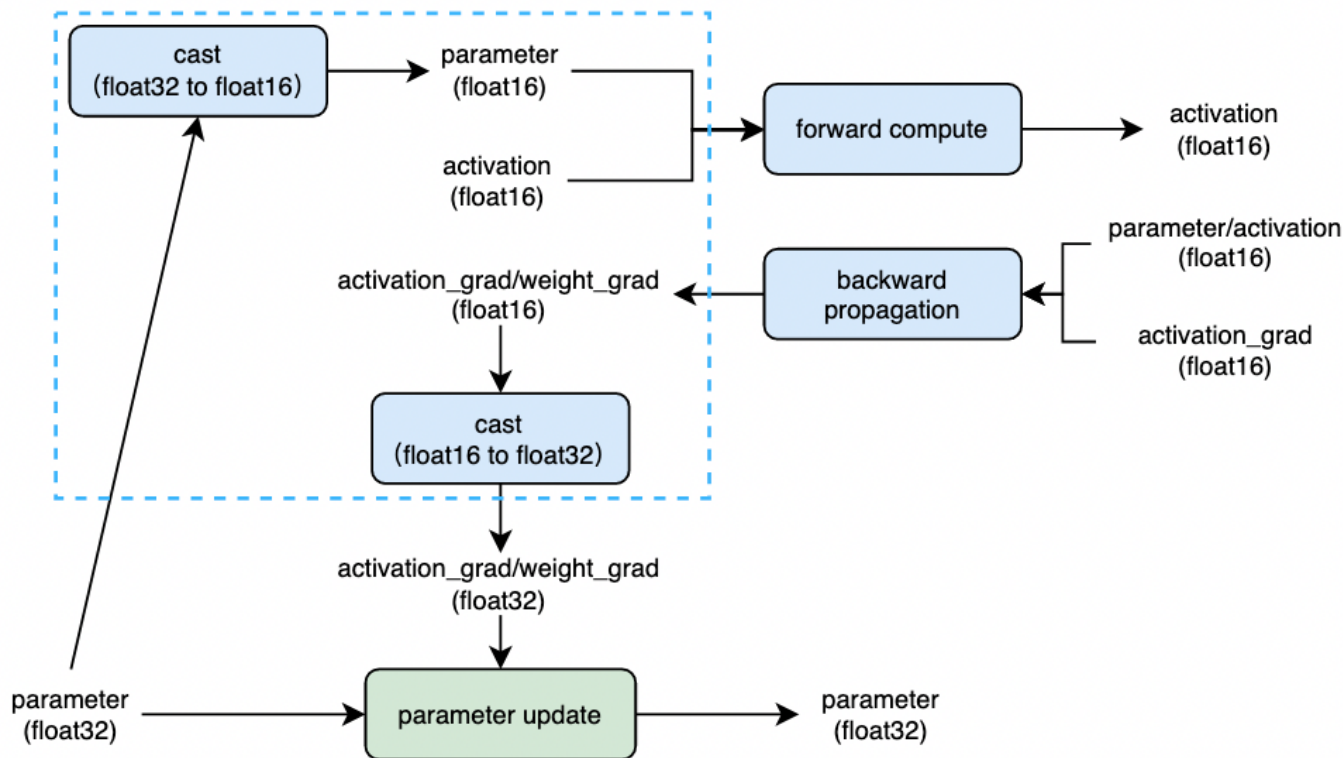
混合精度训练是大模型训练的重要技术之一，一方面能减少模型的显存占用，另一方面由于硬件特性可获得优良的性能加速效果。

飞桨全面支持业界主流自动混合精度训练技术，具体地：

- 支持FP16和BF16等多种低精度训练类型。对于FP16训练，极易发生上溢和下溢现象，飞桨支持了Dynamic Loss Scaling的训练策略，动态调整Loss Scale的值，保证模型正常收敛；对于BF16训练，一般不会发生溢出现象，飞桨支持Static Loss Scaling（甚至不做Loss Scaling）的策略，保证训练过程不会因为Loss Scale过大导致不收敛的现象。

- 支持O1和O2等多种混合精度训练策略，用于可通过黑白名单的方式决定哪些OP以低精度运行。对于O1策略，梯度同步和参数更新均是以FP32进行的；对于O2策略，梯度同步和参数更新均是以FP16进行的，但会额外保存一份Master Weight参数，保证模型训练后期不会因为梯度值过小而不做参数更新。

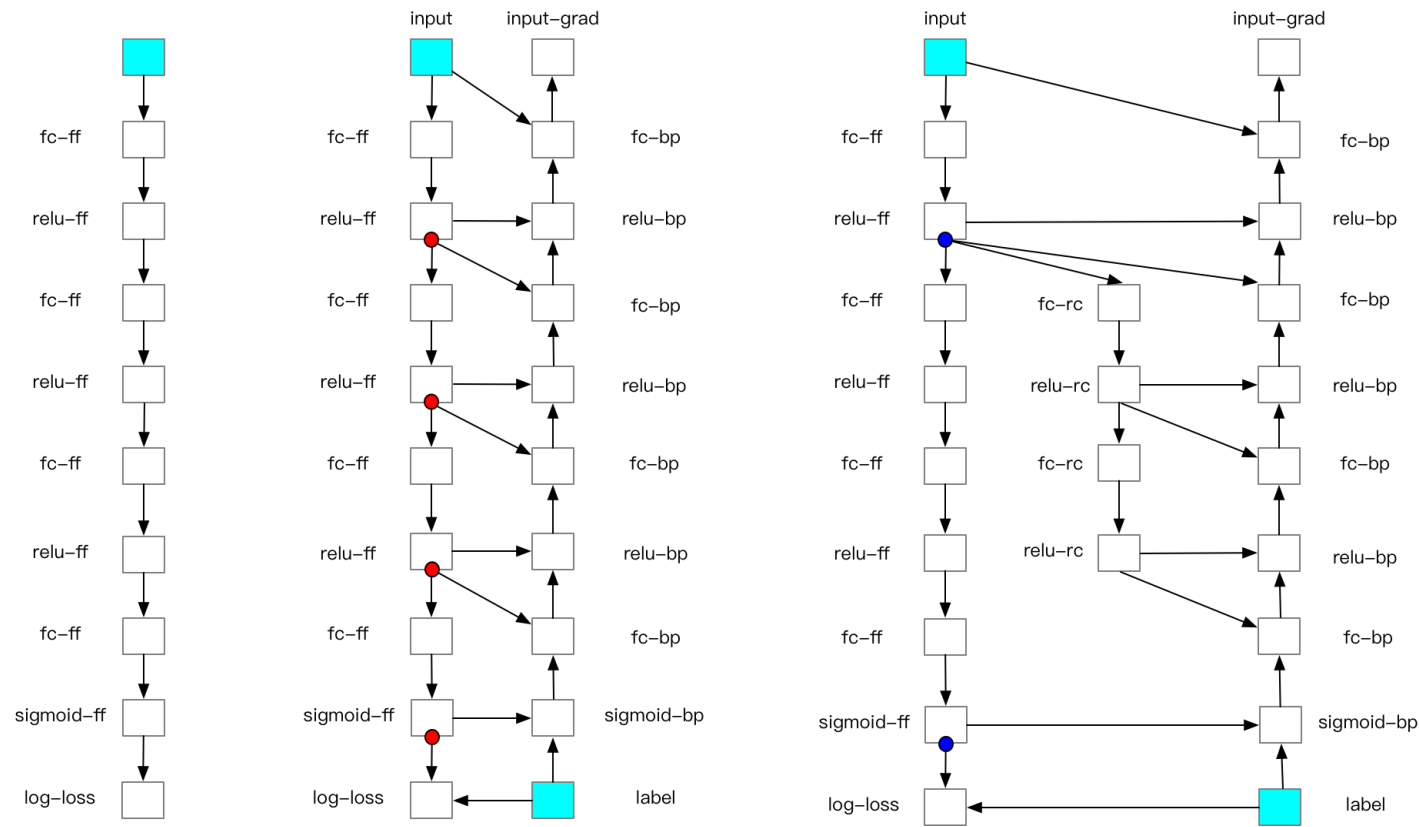
与此同时，飞桨实现了一系列高效的低精度算子，同时结合硬件特性集成了硬件相关的高性能低精度运算库（cuDNN、cuBLAS等）。



混合精度训练流程

六、重计算技术

除了模型参数以外，在大模型训练过程中会产生大量的临时变量，占用大量的显存空间。重计算技术通过反向重新计算前向的方式，极大地压缩临时变量的显存占用，保障大模型训练的高效进行。飞桨提供了通用的重计算接口 `paddle.distributed.fleet.utils.recompute`，方便用户实现大模型训练。值得注意的是，在重计算过程中，如果遇到前向带有随机性的情况（如Dropout等算子），需要在反向使用相同的随机数种子进行计算。飞桨的重计算接口内部已对其进行自动支持，用户无需关注内部实现细节即可安全使用。



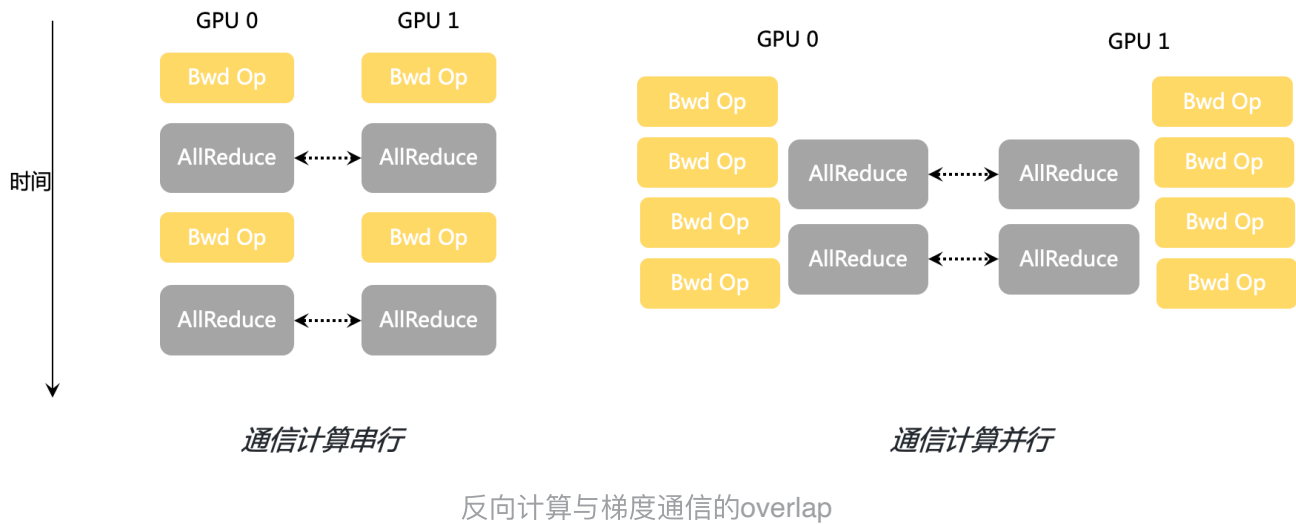
七、梯度累加技术

梯度累加技术是常见的增大batch size、节省显存的训练技术，飞桨对此策略进行了全面支持。梯度累加的原理是将mini-batch拆分为若干个micro-batch，并逐一将mirco-batch运行网络前反向，然后对多个micro-batch的数据的反向梯度进行累加，再合并执行一次参数更新操作。假设梯度累加的步长为K（即将mini-batch拆分为几个micro-batch），在分布式训练过程中，前K-1个micro-batch的反向梯度无需进行AllReduce通信，只有到了第K个micro-batch反向计算后才进行AllReduce通信，可有效降低多卡通信量，提升训练性能。

八、分布式策略加速

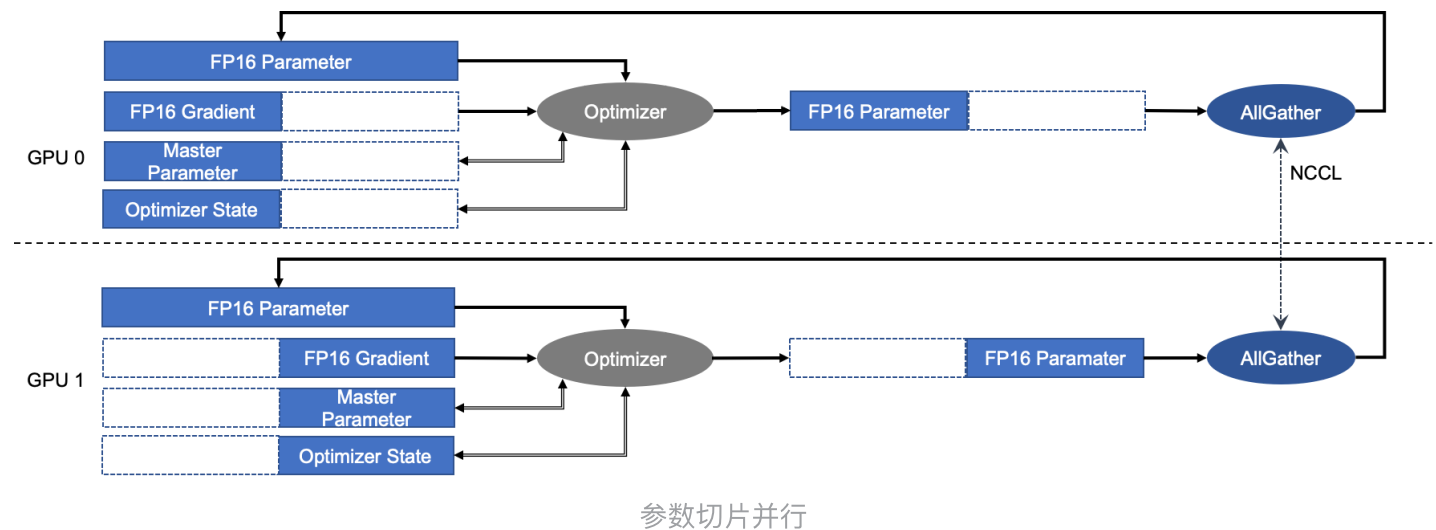
1. 反向计算-通信overlap

在反向计算过程中，参数梯度的AllReduce通信可与反向算子计算在时间线上进行overlap，掩盖通信开销，提高分布式训练加速比。

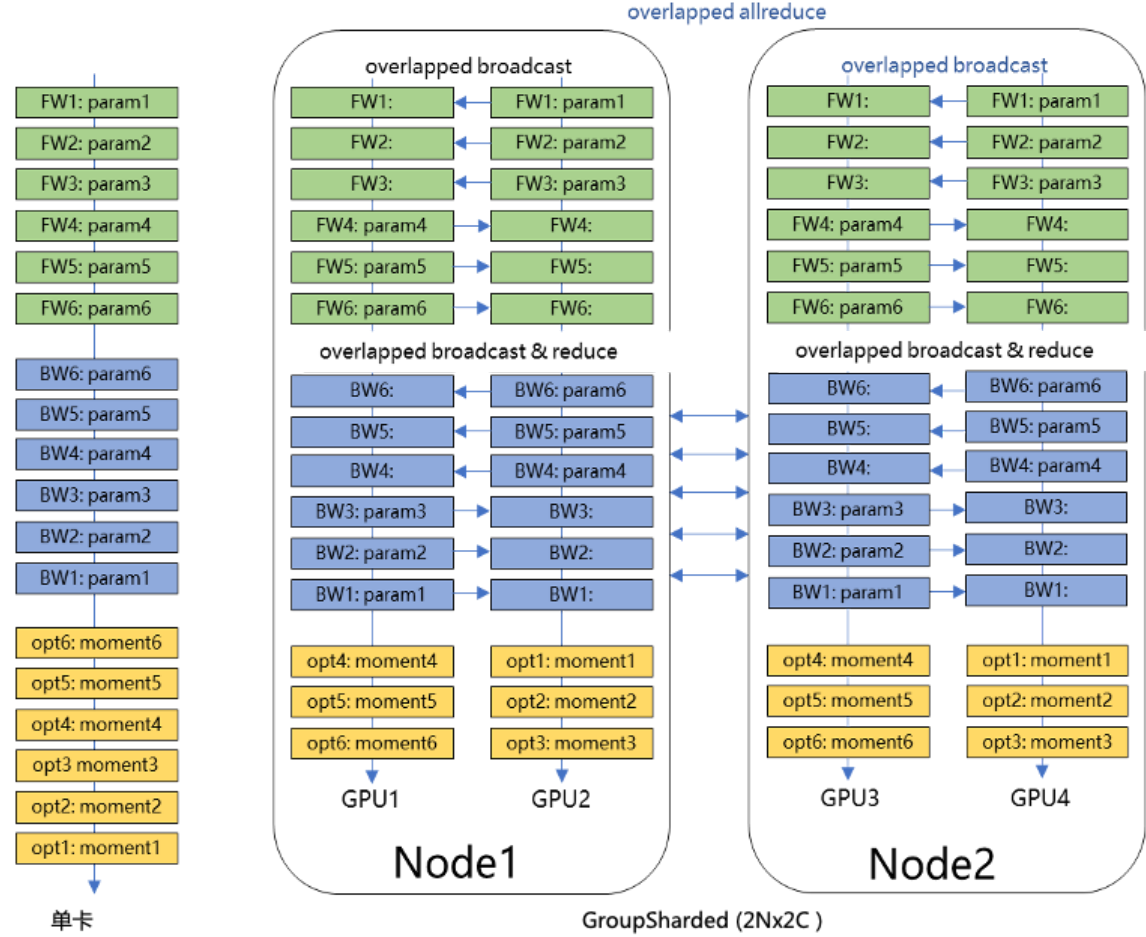


2. 分组参数切片并行

飞桨支持参数切片并行训练策略，支持将Optimizer States (Sharding Stage 1)、参数梯度 (Sharding Stage 2)、参数切分 (Sharding Stage 3) 切分到不同设备上，降低显存占用，支持百亿级模型高效训练。在部分场合下，飞桨的参数切片并行技术可达到超线性加速。

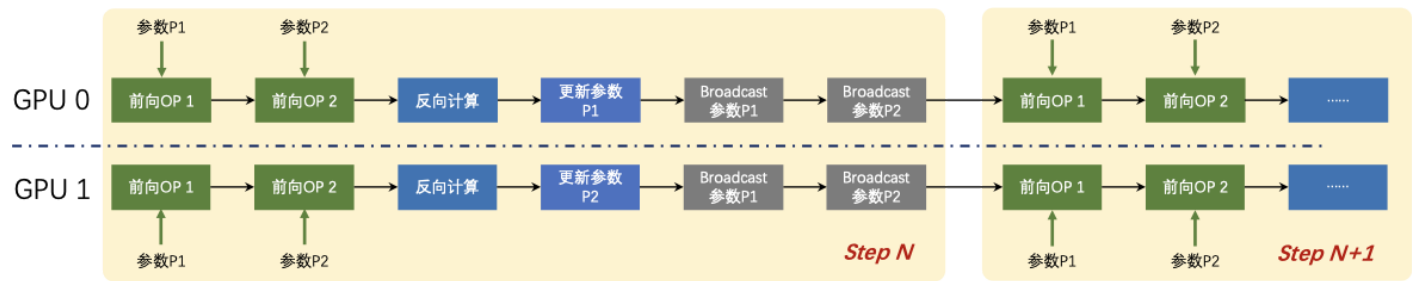


在此基础上，飞桨支持Sharding + DP的分组参数切片并行策略，组内Sharding实现参数切片降低显存占用，组间数据并行提升模型吞吐。在部分场合下，扩展效率可达94%以上。

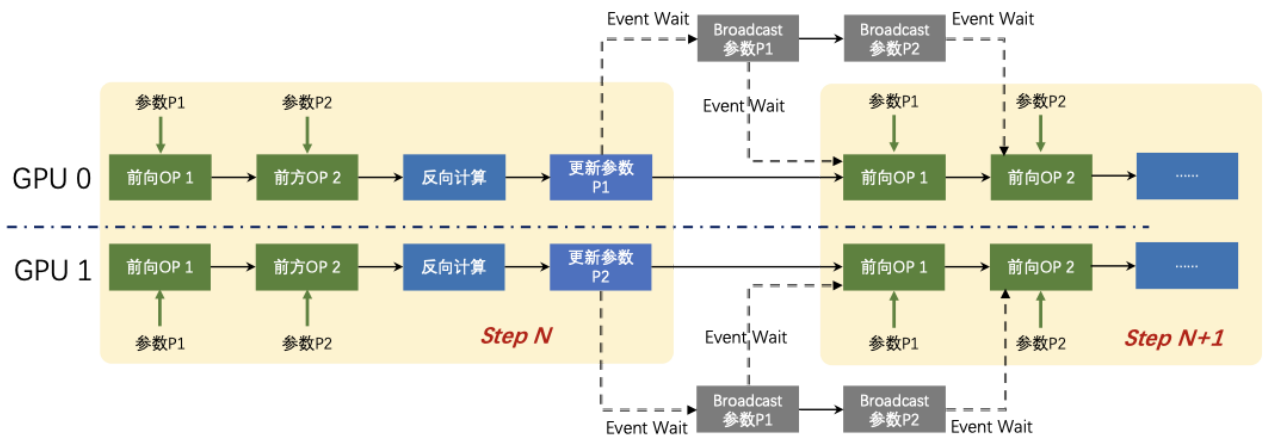


飞桨的分组参数切片并行技术

参数切片并行过程中，每个卡更新完各自的部分参数后，需要将其broadcast（或allgather）到其他卡上去。飞桨创新提出了跨step的通信-计算overlap，即上一个step的参数广播与下一个step的前向计算做overlap，在下一个step网络计算需要此参数时才触发等待参数广播完成的操作，配合多CUDA Stream计算，可为百亿参数规模模型带来11%以上的性能提升。



跨Step计算-通信重叠 + 多流并发广播



跨step的通信-计算overlap策略 + 多流参数广播优化